Sweeps & Segment Trees in Practice

Compress, sort the events, keep the right structure

Learning goals

- Coordinate-compress values and segment endpoints correctly
- Turn 2D problems into **1D sweep + data structure**
- Order simultaneous events deliberately
- Answer range queries **offline** by sorting them
- Use segment-tree patterns: cover counting, descent, mergeable nodes

Why sweeps at regionals

- "$n \le 2 \cdot 10^5$ rectangles / segments / intervals / points"
- $O(n^2)$ pairs is too slow; the geometry is axis-parallel
- Recipe:
  i. Choose a sweep direction (usually x or time)
  ii. Turn objects into **events**
  iii. Maintain the cross-section in a Fenwick / segment tree
  iv. Answer queries when the sweep line passes them

Coordinate compression

```
vector<int> xs = all_values;
sort(xs.begin(), xs.end());
xs.erase(unique(xs.begin(), xs.end()), xs.end());
auto id = [&](int v) {
    return int(lower_bound(xs.begin(), xs.end(), v) - xs.begin());
};
```

- Keeps **order**, discards distances
- Need distances (lengths, areas)? Index **elementary intervals** `[xs[i], xs[i+1]]` and look up real lengths from `xs`
- Compress queries' coordinates too, or use `upper_bound − 1`

1D warm-up: max overlap

CSES 1619 Restaurant Customers: max number of people simultaneously present.

```
events: (arrive, +1), (leave, −1)
sort by time; at equal time, which first?
→ statement: all times distinct, else decide!
```

- Equal-time rule: closed intervals `[a,b]` → process `+1` before `−1`
- Half-open `[a,b)` → `−1` before `+1`

Event ordering is the problem

Situation	Order at equal coordinate
Touching segments count as intersecting	open → query → close
Touching intervals do *not* overlap	close → open
Query "at time t" includes events at t	update → query

- Encode the order in the event tuple: `(x, type, ...)` with type 0/1/2
- Then plain `sort` does the right thing

Segments intersection count

CSES 1740 Intersection Points: horizontal and vertical segments.

```
sweep x →
horizontal [x1,x2] at y:  at x1 add(y,+1), at x2 add(y,−1)
vertical at x, [y1,y2]:   count active y in [y1,y2]  (Fenwick)
equal x: add (0) → query (1) → remove (2)
```

- O(n log C), C = coordinate range (or compress)

Union area of rectangles

CSES 1741 Area of Rectangles

```
sweep x; between consecutive event x's:
    area += (covered y-length) × Δx
rectangle [x1,x2]×[y1,y2]:  +1 on [y1,y2) at x1, −1 at x2
```

- Need: add ±1 on a y-range, query "length with count > 0"
- Not a standard lazy tree: **cover-count trick**

The cover-count segment tree

```
void pull(int node, int l, int r) {
    if (cnt[node] > 0) len[node] = ys[r+1] - ys[l];   // fully covered
    else if (l == r)   len[node] = 0;
    else len[node] = len[2*node] + len[2*node+1];
}
void update(int node, int l, int r, int ql, int qr, int v) {
    if (qr < l || r < ql) return;
    if (ql <= l && r <= qr) { cnt[node] += v; pull(node, l, r); return; }
    int mid = (l + r) / 2;
    update(2*node, l, mid, ql, qr, v); update(2*node+1, mid+1, r, ql, qr, v);
    pull(node, l, r);
}
```

- No push-down: every `-1` matches an earlier `+1` on the **same** nodes

Area sweep: main loop

```
sort(ev.begin(), ev.end());                // (x, ±1, y1, y2)
for (size_t i = 0; i < ev.size(); i++) {
    if (i > 0) area += len[1] * (ev[i][0] - ev[i-1][0]);
    int a = id(ev[i][2]), b = id(ev[i][3]);
    if (a < b) update(1, 0, M - 1, a, b - 1, ev[i][1]);
}
```

- Leaf i = elementary interval `[ys[i], ys[i+1]]`, so update `[a, b−1]`
- Area can reach $4 \cdot 10^{12}$ → `long long`

Offline queries: sort them too

CSES 1734 Distinct Values Queries: number of distinct values in `[a, b]`.

- Sort queries by right end `b`; sweep `r = 1..n`
- Keep a 1 only at the **last occurrence** of each value seen so far
- Then answer = sum of `[a, b]` in a Fenwick tree

```
x:      3 2 3 1 2        at r = 5:
marks:  0 0 1 1 1        query [2,4]? (needs r = 4: 0 1 1 1 → 3)
```

Distinct values: code core

```
for (auto [b, a, id] : qs) {                    // sorted by b
    while (r < b) {
        r++;
        auto it = lastPos.find(x[r]);
        if (it != lastPos.end()) fw.add(it->second, -1);
        lastPos[x[r]] = r;
        fw.add(r, +1);
    }
    ans[id] = fw.sum(b) - fw.sum(a - 1);
}
```

- Same idea: CF 1000F *One Occurrence* (segment tree instead of Fenwick)

Nested intervals

CSES 2168/2169 Nested Ranges: does range i contain / is it contained in another?

- Sort by left ascending, and for equal left by right **descending**
- Then "j contains i" ⇒ j comes before i
- Count earlier ranges with right ≥ r_i → Fenwick over compressed rights
- Tie-breaking in the sort is the whole difficulty

Pattern: walking down the tree

CSES 1143 Hotel Queries: first hotel with ≥ r free rooms.

```
int takeFirst(int node, int l, int r, int need) {
    if (mx[node] < need) return -1;
    if (l == r) { mx[node] -= need; return l; }
    int mid = (l + r) / 2;
    int res = mx[2*node] >= need ? takeFirst(2*node, l, mid, need)
                                 : takeFirst(2*node+1, mid+1, r, need);
    mx[node] = max(mx[2*node], mx[2*node+1]);
    return res;
}
```

- O(log n) instead of binary search × query = O(log² n)

Pattern: mergeable nodes

- Store a small **struct** per node and a `merge(L, R)` function
- Max subarray sum: `(sum, best prefix, best suffix, best)`

```
Node merge(Node a, Node b) {
    return { a.sum + b.sum,
             max(a.pre, a.sum + b.pre),
             max(b.suf, b.sum + a.suf),
             max({a.best, b.best, a.suf + b.pre}) };
}
```

- Also: longest run of equal values, bracket balance, count of inversions in a window

Pattern: segment tree over values

- Index the tree by (compressed) **value**, not position
- "k-th smallest", "how many ≤ x", "first value with property"
- Combined with a sweep over positions → 2D dominance counting
- *CF 1311F Moving Points*: sort by x, Fenwick over compressed speeds

Choosing the structure

Need	Use
Prefix sums with point updates	Fenwick
Range min/max, static	sparse table
Range add + range sum / min	lazy segment tree
Union length / area	cover-count tree
First index with property	segment tree descent
Ordered set operations	`std::set` / PBDS

Complexity

Problem	Time
Max overlap	O(n log n)
Segment intersections	O(n log n)
Union of rectangles	O(n log n)
Distinct values offline	O((n + q) log n)
Hotel queries	O((n + m) log n)

Common pitfalls

- Equal-coordinate events in the wrong order
- Compressing points instead of **elementary intervals** for lengths
- Off-by-one: leaf i covers `[ys[i], ys[i+1])`; update `[a, b−1]`
- `int` area or count overflow
- Forgetting to compress query coordinates
- Fenwick indices starting at 0

Practice problems

- CSES 1619 Restaurant Customers · 1740 Intersection Points · 1741 Area of Rectangles
- CSES 1734 Distinct Values Queries · 1143 Hotel Queries · 1144 Salary Queries
- CSES 2168 / 2169 Nested Ranges Check / Count · 1190 Subarray Sum Queries
- Codeforces 610D — Vika and Segments
- Codeforces 1000F — One Occurrence
- Codeforces 652D — Nested Segments
- Codeforces 1311F — Moving Points
- AtCoder ABC 221 D — Online games

Summary

- Sweep = sorted events + a data structure for the cross-section
- Event order at equal coordinates encodes open/closed semantics
- Offline queries: sort them and sweep alongside the data
- Segment tree tricks: cover counts, descent, mergeable structs, value-indexed
- Next: number theory for regionals